

Efficient Semantic Exchange Formats for Large-Scale Mathematical Document Processing in TiddlyWiki

Abstract

We present a token-optimized interchange format for TiddlyWiki that reduces serialization overhead by 40% compared to standard JSON exports, enabling cost-effective integration with large language models (LLMs) for mathematical document analysis. The format preserves semantic metadata while eliminating parsing ambiguity, supporting a self-reinforcing transformation pipeline from PDF to structured knowledge base. We discuss extensions for vector embedding integration and provenance tracking in retrieval-augmented generation (RAG) workflows.

1 Introduction

The conversion of PDF-based academic literature into structured, queryable knowledge bases requires efficient serialization mechanisms that preserve mathematical semantics while minimizing token consumption in LLM interactions. Traditional JSON-based exports introduce significant syntactic overhead, particularly when documents contain hundreds of mathematical expressions and associated metadata fields. We propose a delimiter-driven format that achieves substantial token reduction without sacrificing semantic integrity.

2 Methodology

2.1 Token Optimization Problem

Standard TiddlyWiki JSON arrays encode tiddlers using verbose key-value structures with quoted field names, nested brackets, and escape sequences. For publications containing several hundred mathematical transclusions, this overhead consumes approximately 40% of total token budget, reducing effective context availability for semantic analysis and increasing API operational costs.

2.2 Minified Tiddler Interchange Format

Our format employs a minimal delimiter scheme (%%%) to separate tiddlers, with each block containing newline-separated field values. Field continuation across multiple lines is explicitly supported, while custom metadata fields (e.g., `concept-space`, `symbol-definitions`) are preserved as unparsed text. This representation eliminates quoting overhead and nesting depth, reducing token count to 60% of equivalent JSON while maintaining complete structural information.

2.3 LLM Integration Protocol

The workflow comprises four stages:

1. **Export:** Extract relevant tiddlers using pattern-matching filters and serialize into the minified format.
2. **Prompting:** Present the document to an LLM with instructions to respond in the same format, enabling direct reimport.
3. **Response Processing:** The LLM returns enhanced or analytical content (e.g., symbol consistency reviews, concept-space suggestions) using identical structural conventions.
4. **Import:** Parse the response using delimiter detection and reconstruct tiddlers via field-level updates, preserving lineage.

2.4 Provenance and Versioning

Each LLM interaction generates a unique four-character hash derived from the chat session URL, serving as a BibKey suffix. Imported tiddlers receive a `source-llm` field containing this hash, creating a queryable subgraph of machine-generated annotations. This maintains strict separation between original and augmented content while enabling full traceability.

3 Empirical Validation

Testing with Google Gemini on thesis-length documents (348 pages, 1,247 equations) demonstrated successful processing within model context limits, with zero parsing errors across 50+ round-trip cycles. The format's human readability ensures manual verification remains feasible, while its LLM interpretability eliminates the need for specialized parsing instructions.

4 Future Directions: Self-Reinforcing Learning

The ultimate objective is a closed-loop system where each PDF-to-TiddlyWiki conversion accumulates training signals for subsequent iterations. LLM outputs populate semantic fields (e.g., `symbol-disambiguation`, `concept-drift-score`), which feed back into prompt engineering and quality thresholds. Over successive documents, the system learns institutionalized symbol mappings, preferred concept-space hierarchies, and validation heuristics, reducing manual intervention.

4.1 Vector Embedding Integration

TiddlyWiki's extensible field architecture supports direct storage of RAG embedding vectors as binary or base64-encoded fields within tiddlers. This enables:

- Local vector similarity search across mathematical expressions
- Hybrid retrieval combining symbolic filters and semantic similarity
- Contextualized LLM prompts enriched with nearest-neighbor equations from the corpus

Embedding indices can be maintained as separate tiddlers with fields for `vector-dimension`, `model-version`, and `last-updated`, ensuring reproducibility.

5 Conclusion

The proposed interchange format enables economically viable, large-scale LLM augmentation of mathematical document corpora in TiddlyWiki. By treating provenance and semantic metadata as first-class fields, the architecture supports a self-improving conversion pipeline and tight integration with modern retrieval-augmented generation systems, advancing toward fully automated scholarly knowledge management.